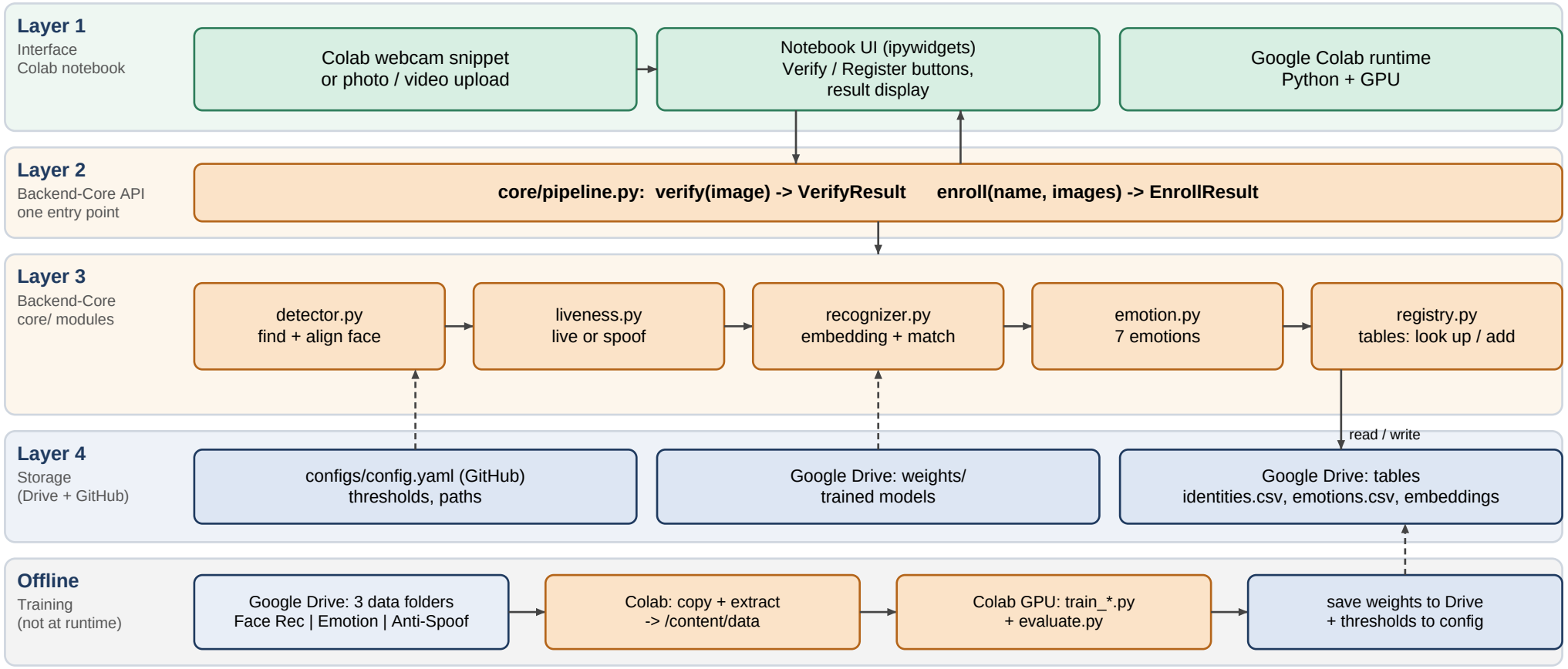
1. Overall architecture

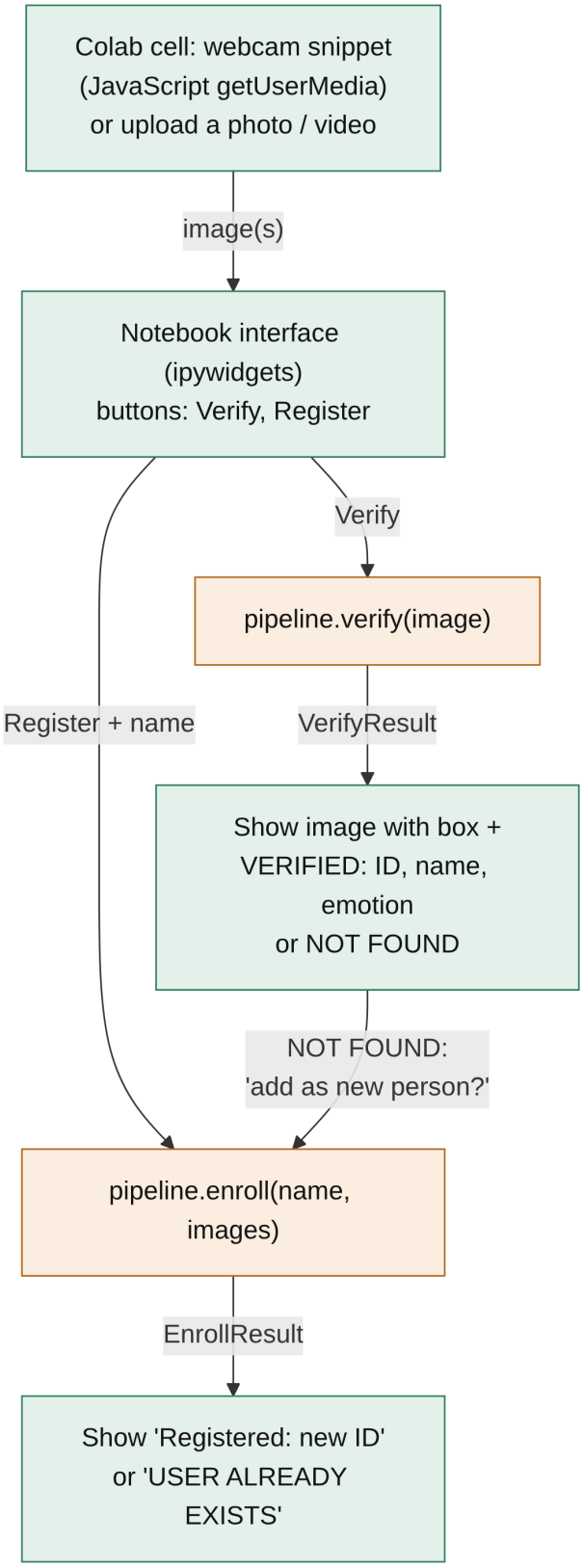
Python only, running in Google Colab. Data, models and the lookup tables live on Google Drive.



- **Layer 1 Interface:** a Colab notebook (webcam snippet or photo upload + ipywidgets buttons). No separate desktop app needed.
- **Layer 2 pipeline.py:** the only public API (verify, enroll).
- **Layer 3 core modules:** detector, liveness, recognizer, emotion, registry; one job each. Liveness is a gate: spoofed faces are never recognized.
- **Layer 4 storage:** Google Drive holds the data (three sub-folders: Face Recognition, Emotion Detection, Anti-Spoofing) plus weights, tables and embeddings. Code and config.yaml are on GitHub.
- **Offline training:** Colab GPU reads each data folder from Drive, trains one model per folder, saves weights back to Drive.

2. Interface layer (Colab notebook)

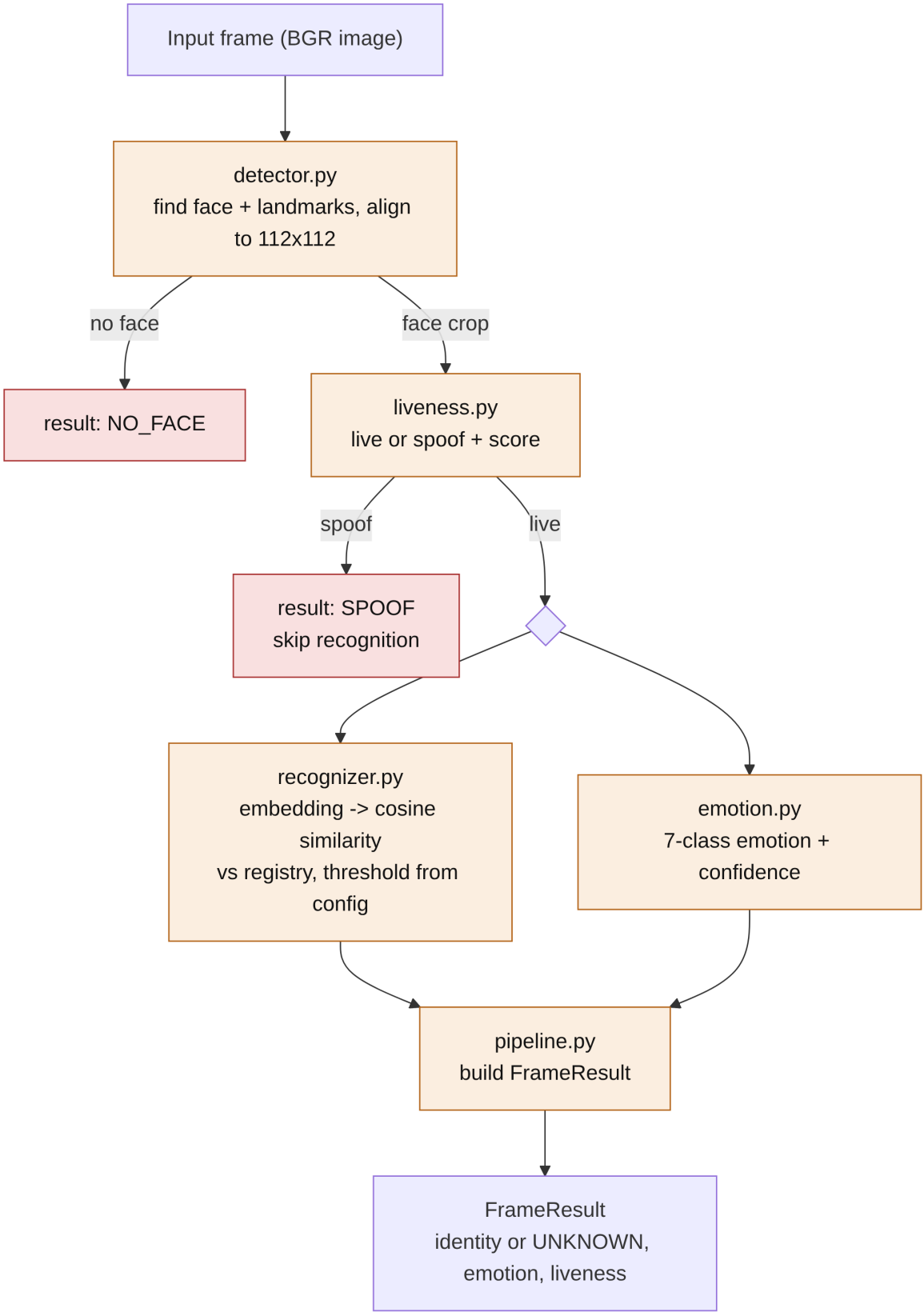
Colab cell for the camera, ipywidgets for buttons. No model code here.



- Colab cannot open a laptop camera directly; a small JavaScript snippet (getUserMedia) captures a photo, or the user uploads a photo or video.
- Verify sends the image to pipeline.verify(); Register sends a name and several images to pipeline.enroll().
- Result is shown as an image with a face box and text: VERIFIED, NOT FOUND, REGISTERED or USER ALREADY EXISTS.

3. Core pipeline (per image)

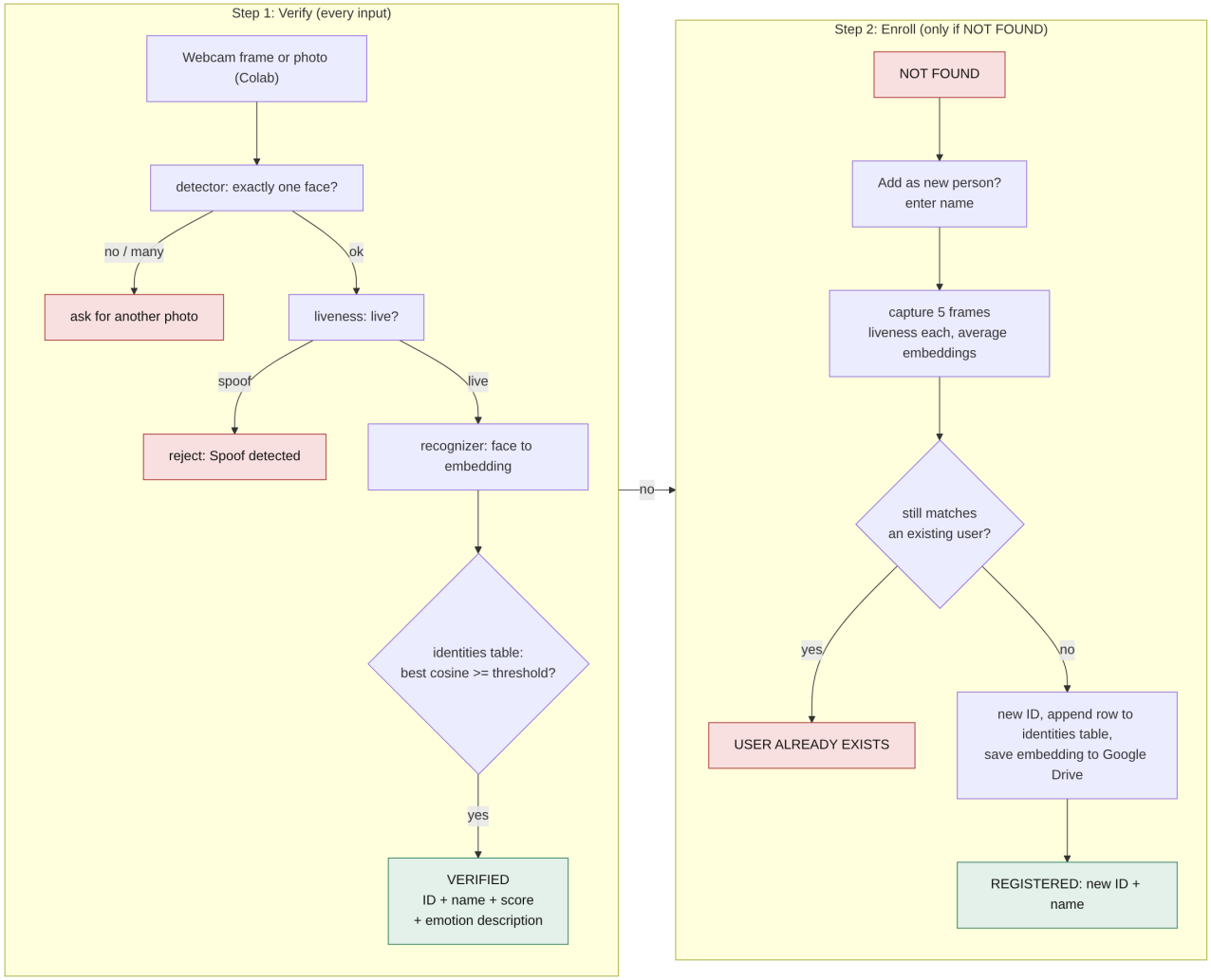
Liveness is a gate: a spoofed face is never recognized.



- **detector** finds and aligns the face. No face means stop early.
- **liveness** runs first. Spoof means skip recognition and emotion.
- **recognizer** and **emotion** run only on live faces; results are merged into one result object.
- Every module takes and returns plain data (schemas.py), so each can be tested alone.

4. Verify, then enroll if not found

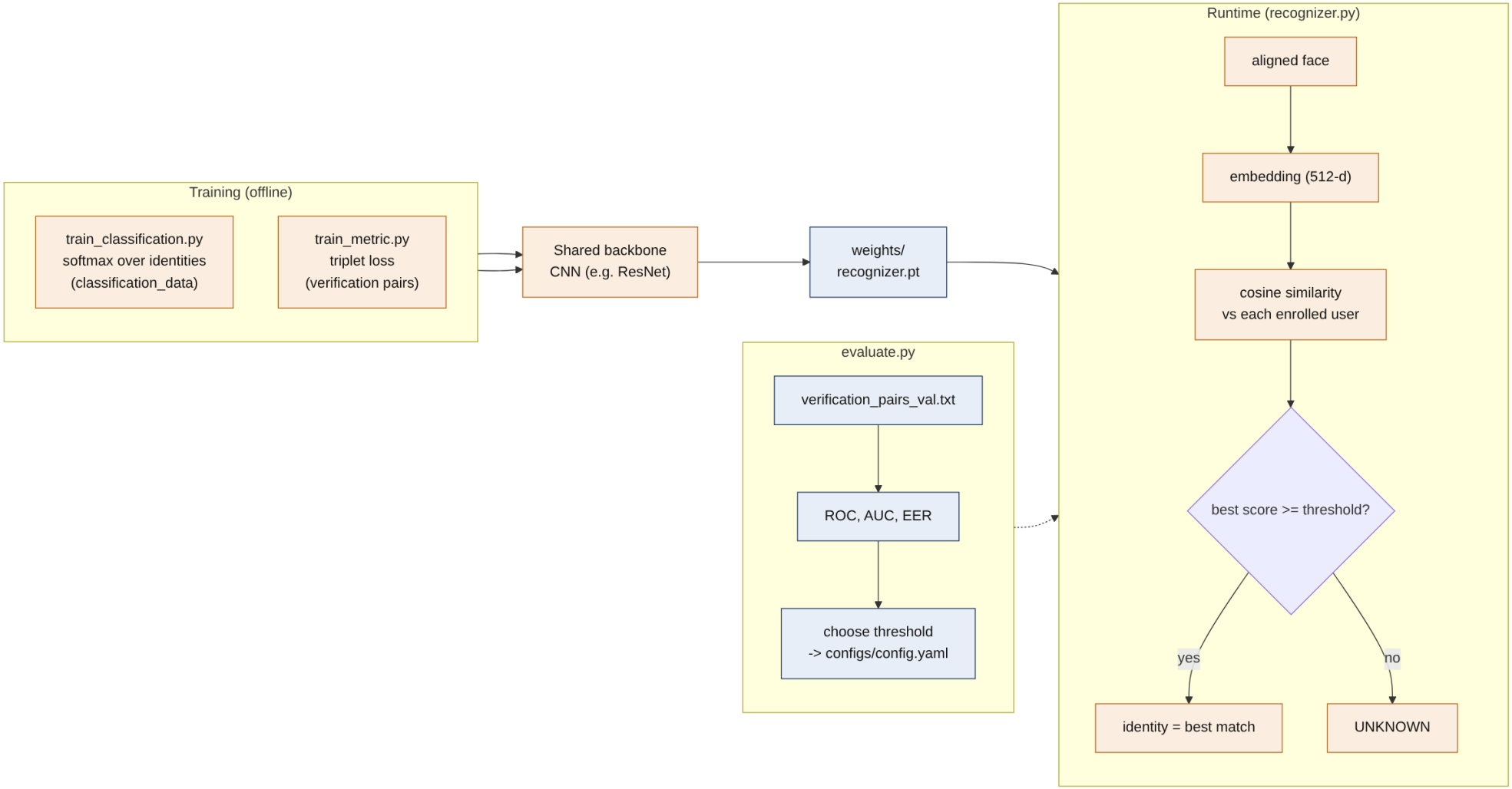
Search the identities table first; add a new person only when nothing matches.



- **Verify:** the embedding is compared with every row of the identities table. Best cosine score above the threshold = VERIFIED with ID and name.
- **Emotion:** the emotion model outputs an ID; the emotions table turns it into a description.
- **Enroll:** only after NOT FOUND. Capture 5 frames, check liveness, average embeddings, re-check for duplicates, then append a new row and save the embedding.

5. Recognition model layer

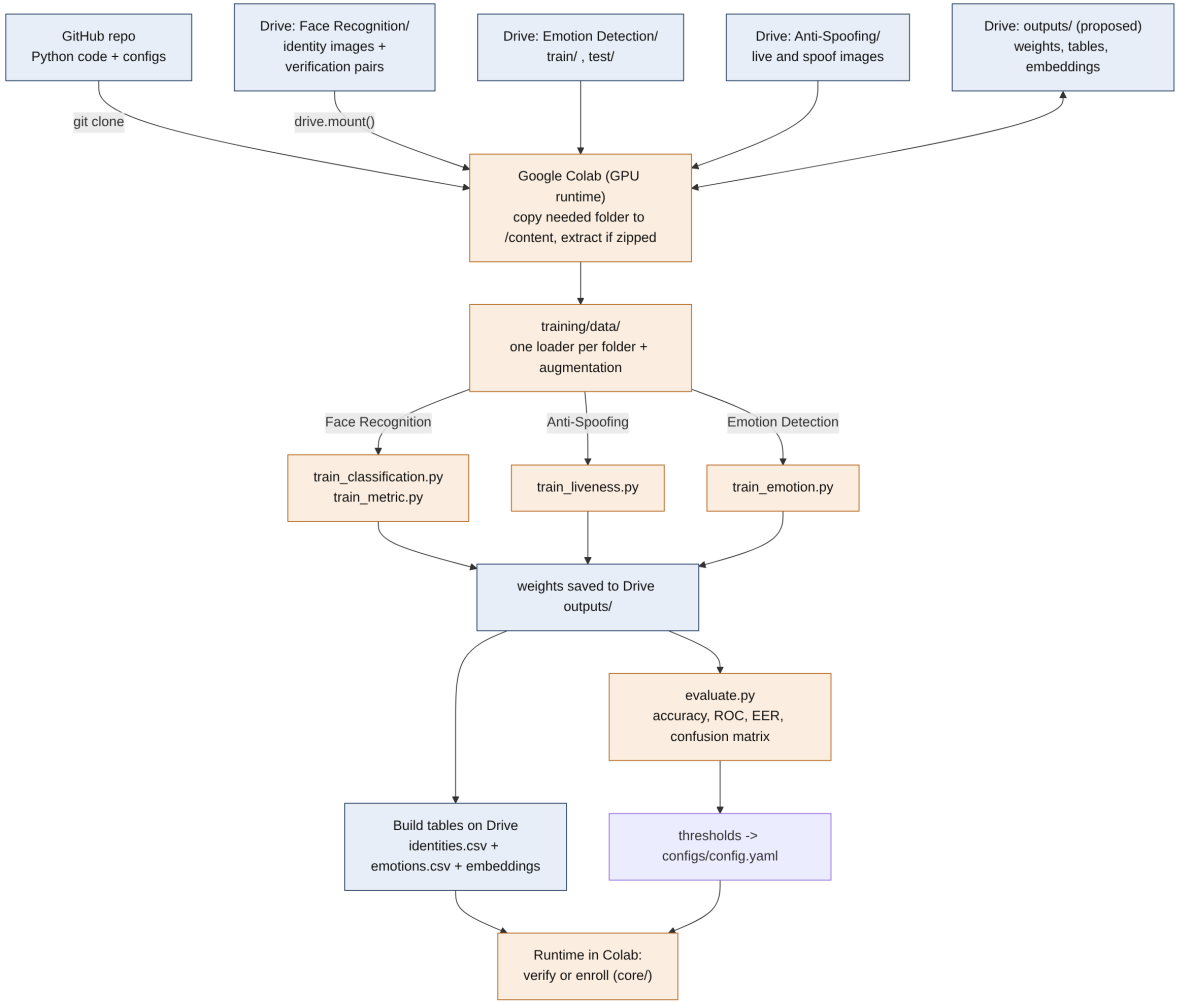
Train with two methods, deploy one embedding model, tune the threshold on validation pairs.



- **Classification (softmax)** and **metric learning (triplet loss)** are compared; both feed the same backbone.
- At runtime a face becomes an embedding, compared by cosine similarity with enrolled users.
- **evaluate.py** computes ROC, AUC and EER on the validation pairs and picks the threshold stored in config.yaml.

6. Data, Colab and training layer

Three data folders on Google Drive, one per task; Python code from GitHub; training and runtime run in Colab.



- Drive has three data folders: **Face Recognition** (-> classification and metric models), **Emotion Detection** (train/ and test/ -> emotion model), **Anti-Spoofing** (-> liveness model). Contents of the first and last are still to be confirmed.
- Mount Drive, copy the folder you need to /content (much faster than reading from Drive file by file). Weights are saved back to Drive so they survive Colab resets.
- After training, build the tables on Drive: identities.csv, emotions.csv and the embeddings file.

7. Lookup tables (on Google Drive)

Two small tables decide what the system prints. Identities grows; emotions is fixed.

identities.csv (grows)

ID	Name	Image	Status	embedding_file
0001	Person A	img/0001.jpg	old member	emb/0001.npy
0002	Person B	img/0002.jpg	old member	emb/0002.npy
...	...	...	...	...
0501	Person X	img/0501.jpg	new member	emb/0501.npy

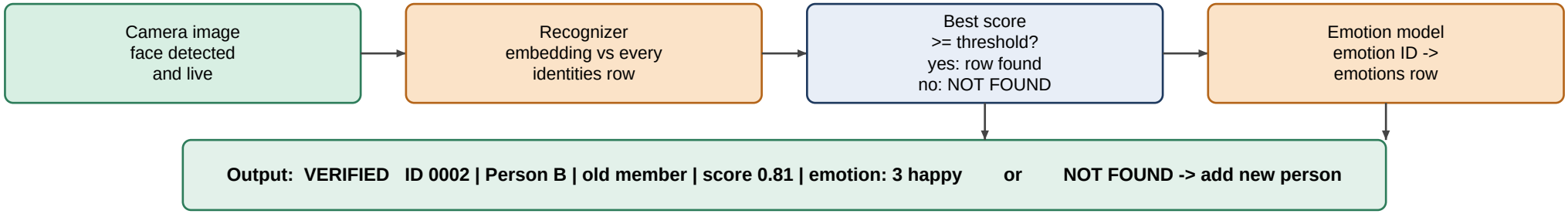
Old member = from the training data. New member = enrolled while running.

emotions.csv (fixed)

ID	label	description
0	angry	Brows lowered, tense mouth
1	disgust	Nose wrinkled, upper lip raised
2	fear	Eyes wide, brows raised
3	happy	Smile, raised cheeks
4	neutral	No strong expression
5	sad	Mouth corners down
6	surprise	Mouth open, brows up

Example only: use the real emotion labels of the dataset.

How a result is built



- **identities.csv** columns: ID, Name, Image (one reference photo path), Status (old member = from the training data, new member = enrolled at runtime) and embedding_file (what matching really uses).
- **emotions.csv** is fixed. The emotion model predicts an ID; the table gives its description. Use the real labels of the dataset (example shown).
- VERIFIED output = identities row (ID, Name, Status) + cosine score + emotions row (ID, description). Photos, tables and embeddings stay on Drive (outputs/ folder, proposed), never in GitHub: they are personal data.